

资源受限铁道知识图片描述软件源代码

软件名称：资源受限铁道知识图片描述软件 V1.0

版本号：V1.0

文档类型：源代码

目录

1. src/railcap/__init__.py
2. src/railcap/benchmark.py
3. src/railcap/config.py
4. src/railcap/evaluate.py
5. src/railcap/knowledge/__init__.py
6. src/railcap/knowledge/constraints.py
7. src/railcap/knowledge/ontology.py
8. src/railcap/metrics/__init__.py
9. src/railcap/metrics/domain_metrics.py
10. src/railcap/models/__init__.py
11. src/railcap/models/compression.py
12. src/railcap/models/distillation.py
13. src/railcap/models/modeling.py
14. src/railcap/predict.py
15. src/railcap/train.py
16. src/railcap/utils/__init__.py
17. src/railcap/utils/runtime.py
18. pyproject.toml
19. configs/default.yaml
20. tests/test_domain_metrics.py
21. tests/test_ontology.py

src/railcap/__init__.py

```
"""Railway knowledge-guided lightweight image captioning."""

__version__ = "0.1.0"
```

src/railcap/benchmark.py

```
from __future__ import annotations

import argparse
import json
import time
from pathlib import Path

import torch
from torch.utils.data import DataLoader
from transformers import AutoImageProcessor, AutoTokenizer, VisionEncoderDecoderModel

from railcap.config import load_config
from railcap.data.dataset import RailwayCaptionDataset
from railcap.train import collate_batch
from railcap.utils.runtime import cuda_memory_mb, pick_device


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Benchmark caption generation latency.")
    parser.add_argument("--config", default="configs/default.yaml", type=Path)
    parser.add_argument("--checkpoint", type=Path, help="Model checkpoint directory.")
    parser.add_argument("--manifest", type=Path, help="Override benchmark manifest.")
    parser.add_argument("--warmup", type=int, default=3)
    parser.add_argument("--steps", type=int, default=20)
    return parser.parse_args()


def main() -> None:
    args = parse_args()
    cfg = load_config(args.config)
    raw = cfg.raw

    checkpoint = args.checkpoint or (cfg.output_dir / "final")
    manifest_path = args.manifest or Path(raw["data"])[ "test_manifest" ])
    device = pick_device()

    image_processor = AutoImageProcessor.from_pretrained(checkpoint)
    tokenizer = AutoTokenizer.from_pretrained(checkpoint)
    model = VisionEncoderDecoderModel.from_pretrained(checkpoint).to(device)
    model.eval()

    dataset = RailwayCaptionDataset(
        manifest_path,
        image_root=raw["data"].get("image_root", "."),
        image_processor=image_processor,
        tokenizer=None,
    )
    loader = DataLoader(
        dataset,
        batch_size=int(raw.get("eval", {}).get("batch_size", 8)),
        shuffle=False,
        num_workers=0,
        collate_fn=collate_batch,
    )

    generation_kwargs = dict(raw.get("model", {}).get("generation", {}))
    timings: List[float] = []
    images = 0

    with torch.no_grad():
```

```

for step, batch in enumerate(loader):
    if step >= args.warmup + args.steps:
        break
    pixel_values = batch["pixel_values"].to(device)
    if device.type == "cuda":
        torch.cuda.synchronize()
    start = time.perf_counter()
    generated_ids = model.generate(pixel_values=pixel_values, **generation_kwargs)
    tokenizer.batch_decode(generated_ids, skip_special_tokens=True)
    if device.type == "cuda":
        torch.cuda.synchronize()
    elapsed = time.perf_counter() - start
    if step >= args.warmup:
        timings.append(elapsed)
        images += pixel_values.shape[0]

total_time = sum(timings)
result = {
    "device": str(device),
    "steps": len(timings),
    "images": images,
    "batch_size": int(raw.get("eval", {}).get("batch_size", 8)),
    "latency_ms_per_batch": (total_time / len(timings) * 1000) if timings else 0.0,
    "latency_ms_per_image": (total_time / images * 1000) if images else 0.0,
    "throughput_img_s": (images / total_time) if total_time else 0.0,
    "cuda_max_memory_mb": cuda_memory_mb(),
}
print(json.dumps(result, ensure_ascii=False, indent=2))

if __name__ == "__main__":
    main()

```

src/railcap/config.py

```

from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from typing import Any

import yaml

@dataclass(frozen=True)
class ExperimentConfig:
    raw: dict[str, Any]
    path: Path

    @property
    def output_dir(self) -> Path:
        return Path(self.raw.get("output_dir", "outputs/default"))

def load_config(path: str | Path) -> ExperimentConfig:
    config_path = Path(path)
    with config_path.open("r", encoding="utf-8") as f:
        raw = yaml.safe_load(f) or {}
    return ExperimentConfig(raw=raw, path=config_path)

def get_nested(config: dict[str, Any], dotted_key: str, default: Any = None) -> Any:
    current: Any = config
    for key in dotted_key.split("."):
        if not isinstance(current, dict) or key not in current:
            return default
        current = current[key]
    return current

```

src/railcap/evaluate.py

```
from __future__ import annotations

import argparse
import json
from pathlib import Path

from railcap.config import load_config
from railcap.data.dataset import load_manifest
from railcap.knowledge.ontology import RailwayOntology
from railcap.metrics.domain_metrics import compute_domain_metrics


def load_predictions(path: str | Path) -> List[str]:
    predictions: List[str] = []
    with Path(path).open("r", encoding="utf-8") as f:
        for line_no, line in enumerate(f, start=1):
            line = line.strip()
            if not line:
                continue
            item = json.loads(line)
            if "prediction" not in item:
                raise ValueError(f"{path}:{line_no}: missing prediction field")
            predictions.append(str(item["prediction"]))
    return predictions


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Evaluate railway caption predictions.")
    parser.add_argument("--config", default="configs/default.yaml", type=Path)
    parser.add_argument("--manifest", type=Path, help="Override test manifest path.")
    parser.add_argument("--predictions", required=True, type=Path)
    return parser.parse_args()


def main() -> None:
    args = parse_args()
    cfg = load_config(args.config)
    raw = cfg.raw

    manifest_path = args.manifest or Path(raw["data"]["test_manifest"])
    samples = load_manifest(manifest_path)
    predictions = load_predictions(args.predictions)
    ontology = RailwayOntology.from_yaml(raw["knowledge"]["ontology_path"])

    metrics = compute_domain_metrics(samples, predictions, ontology)
    print(json.dumps(metrics.as_dict(), ensure_ascii=False, indent=2))

if __name__ == "__main__":
    main()
```

src/railcap/knowledge/__init__.py

```
"""Railway domain knowledge utilities."""
```

src/railcap/knowledge/constraints.py

```
from __future__ import annotations

from dataclasses import dataclass

from railcap.data.schema import CaptionSample
from railcap.knowledge.ontology import RailwayOntology
```

```

@dataclass(frozen=True)
class CaptionKnowledgeReport:
    expected_terms: set[str]
    mentioned_terms: set[str]
    missing_terms: set[str]
    hallucinated_terms: set[str]

    @property
    def has_hallucination(self) -> bool:
        return bool(self.hallucinated_terms)

def expected_terms_from_sample(sample: CaptionSample) -> set[str]:
    return set(sample.equipment) | set(sample.faults) | set(sample.risks)

def analyze_caption(
    sample: CaptionSample,
    generated_caption: str,
    ontology: RailwayOntology,
) -> CaptionKnowledgeReport:
    expected = {ontology.canonicalize(term) for term in expected_terms_from_sample(sample)}
    mentioned = ontology.extract_terms(generated_caption)
    ontology_terms = ontology.canonical_terms

    return CaptionKnowledgeReport(
        expected_terms=expected,
        mentioned_terms=mentioned,
        missing_terms=expected - mentioned,
        hallucinated_terms=(mentioned - expected) & ontology_terms,
    )

def build_generation_prefix(sample: CaptionSample) -> str:
    parts: list[str] = []
    if sample.equipment:
        parts.append("设备: " + "、".join(sample.equipment))
    if sample.faults:
        parts.append("异常: " + "、".join(sample.faults))
    if sample.risks:
        parts.append("风险: " + "、".join(sample.risks))
    if not parts:
        return "请描述铁路巡检图像: "
    return "; ".join(parts) + "。请生成专业、准确、无臆测的铁路图像描述: "

```

src/railcap/knowledge/ontology.py

```

from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path

import yaml

@dataclass(frozen=True)
class RailwayOntology:
    equipment: tuple[str, ...]
    faults: tuple[str, ...]
    risks: tuple[str, ...]
    aliases: dict[str, tuple[str, ...]]

    @classmethod
    def from_yaml(cls, path: str | Path) -> "RailwayOntology":
        with Path(path).open("r", encoding="utf-8") as f:
            raw = yaml.safe_load(f) or {}

        aliases = {
            str(term): tuple(str(alias) for alias in values)

```

```

        for term, values in (raw.get("aliases") or {}).items()
    }
    return cls(
        equipment=tuple(str(x) for x in raw.get("equipment", [])),
        faults=tuple(str(x) for x in raw.get("faults", [])),
        risks=tuple(str(x) for x in raw.get("risks", [])),
        aliases=aliases,
    )

@property
def canonical_terms(self) -> set[str]:
    return set(self.equipment) | set(self.faults) | set(self.risks)

@property
def all_surface_terms(self) -> set[str]:
    terms = set(self.canonical_terms)
    for values in self.aliases.values():
        terms.update(values)
    return terms

def canonicalize(self, term: str) -> str:
    if term in self.canonical_terms:
        return term
    for canonical, aliases in self.aliases.items():
        if term in aliases:
            return canonical
    return term

def extract_terms(self, text: str) -> set[str]:
    found: set[str] = set()
    for term in self.all_surface_terms:
        if term and term in text:
            found.add(self.canonicalize(term))
    return found

```

src/railcap/metrics/__init__.py

```

"""Evaluation metrics for railway captioning."""

```

src/railcap/metrics/domain_metrics.py

```

from __future__ import annotations

from dataclasses import dataclass

from railcap.data.schema import CaptionSample
from railcap.knowledge.constraints import analyze_caption
from railcap.knowledge.ontology import RailwayOntology

@dataclass(frozen=True)
class DomainMetrics:
    equipment_recall: float
    fault_precision: float
    fault_recall: float
    fault_f1: float
    risk_accuracy: float
    hallucination_rate: float

    def as_dict(self) -> dict[str, float]:
        return {
            "equipment_recall": self.equipment_recall,
            "fault_precision": self.fault_precision,
            "fault_recall": self.fault_recall,
            "fault_f1": self.fault_f1,
            "risk_accuracy": self.risk_accuracy,
            "hallucination_rate": self.hallucination_rate,
        }

```

```

def _safe_div(numerator: float, denominator: float) -> float:
    return numerator / denominator if denominator else 0.0

def compute_domain_metrics(
    samples: List[CaptionSample],
    predictions: List[str],
    ontology: RailwayOntology,
) -> DomainMetrics:
    if len(samples) != len(predictions):
        raise ValueError("samples and predictions must have the same length")

    equipment_hits = 0
    equipment_total = 0
    fault_tp = 0
    fault_fp = 0
    fault_fn = 0
    risk_hits = 0
    risk_total = 0
    hallucinated = 0

    for sample, prediction in zip(samples, predictions, strict=True):
        mentioned = ontology.extract_terms(prediction)

        expected_equipment = {ontology.canonicalize(x) for x in sample.equipment}
        equipment_hits += len(expected_equipment & mentioned)
        equipment_total += len(expected_equipment)

        expected_faults = {ontology.canonicalize(x) for x in sample.faults}
        predicted_faults = mentioned & set(ontology.faults)
        fault_tp += len(expected_faults & predicted_faults)
        fault_fp += len(predicted_faults - expected_faults)
        fault_fn += len(expected_faults - predicted_faults)

        expected_risks = {ontology.canonicalize(x) for x in sample.risks}
        if expected_risks:
            risk_total += 1
            if expected_risks & mentioned:
                risk_hits += 1

        report = analyze_caption(sample, prediction, ontology)
        hallucinated += int(report.has_hallucination)

    fault_precision = _safe_div(fault_tp, fault_tp + fault_fp)
    fault_recall = _safe_div(fault_tp, fault_tp + fault_fn)
    fault_f1 = _safe_div(2 * fault_precision * fault_recall, fault_precision + fault_recall)

    return DomainMetrics(
        equipment_recall=_safe_div(equipment_hits, equipment_total),
        fault_precision=fault_precision,
        fault_recall=fault_recall,
        fault_f1=fault_f1,
        risk_accuracy=_safe_div(risk_hits, risk_total),
        hallucination_rate=_safe_div(hallucinated, len(samples)),
    )

```

src/railcap/models/__init__.py

```

"""Model builders and compression helpers."""

```

src/railcap/models/compression.py

```

from __future__ import annotations

from pathlib import Path
from typing import Any

```

```

import torch

def apply_dynamic_quantization(model: torch.nn.Module) -> torch.nn.Module:
    return torch.quantization.quantize_dynamic(
        model,
        {torch.nn.Linear},
        dtype=torch.qint8,
    )

def export_onnx_placeholder(model: torch.nn.Module, output_path: str | Path, example_batch: Any) -> None:
    path = Path(output_path)
    path.parent.mkdir(parents=True, exist_ok=True)
    raise NotImplementedError(
        "ONNX export needs a fixed model/input signature. Add an example pixel_values batch "
        f"and decoder inputs before exporting to {path}."
    )

```

src/railcap/models/distillation.py

```

from __future__ import annotations

import torch
import torch.nn.functional as F

def distillation_kl_loss(
    student_logits: torch.Tensor,
    teacher_logits: torch.Tensor,
    temperature: float = 2.0,
) -> torch.Tensor:
    student_log_probs = F.log_softmax(student_logits / temperature, dim=-1)
    teacher_probs = F.softmax(teacher_logits / temperature, dim=-1)
    return F.kl_div(student_log_probs, teacher_probs, reduction="batchmean") * temperature**2

def combined_distillation_loss(
    ce_loss: torch.Tensor,
    student_logits: torch.Tensor,
    teacher_logits: torch.Tensor,
    alpha_ce: float = 0.7,
    alpha_kl: float = 0.3,
    temperature: float = 2.0,
) -> torch.Tensor:
    kl_loss = distillation_kl_loss(student_logits, teacher_logits, temperature)
    return alpha_ce * ce_loss + alpha_kl * kl_loss

```

src/railcap/models/modeling.py

```

from __future__ import annotations

from typing import Any

import torch
from transformers import (
    AutoImageProcessor,
    AutoTokenizer,
    VisionEncoderDecoderModel,
    VisionEncoderDecoderConfig,
)

def build_processors(config: dict[str, Any]) -> tuple[Any, Any]:
    model_cfg = config["model"]
    if model_cfg.get("pretrained_vision_encoder_decoder"):
        name = model_cfg["pretrained_vision_encoder_decoder"]

```

```

        image_processor = AutoImageProcessor.from_pretrained(name)
        tokenizer = AutoTokenizer.from_pretrained(name)
    else:
        image_processor = AutoImageProcessor.from_pretrained(model_cfg["vision_encoder"])
        tokenizer = AutoTokenizer.from_pretrained(model_cfg["text_decoder"])

    if tokenizer.pad_token is None:
        tokenizer.pad_token = tokenizer.eos_token
    return image_processor, tokenizer

def build_model(config: dict[str, Any], tokenizer: Any) -> VisionEncoderDecoderModel:
    model_cfg = config["model"]
    pretrained = model_cfg.get("pretrained_vision_encoder_decoder")

    if pretrained:
        model = VisionEncoderDecoderModel.from_pretrained(pretrained)
    else:
        decoder_name = model_cfg["text_decoder"]
        model = VisionEncoderDecoderModel.from_encoder_decoder_pretrained(
            model_cfg["vision_encoder"],
            decoder_name,
        )

    model.config.decoder_start_token_id = tokenizer.bos_token_id or tokenizer.eos_token_id
    model.config.eos_token_id = tokenizer.eos_token_id
    model.config.pad_token_id = tokenizer.pad_token_id
    model.config.vocab_size = model.config.decoder.vocab_size

    generation_cfg = model_cfg.get("generation", {})
    for key, value in generation_cfg.items():
        setattr(model.generation_config, key, value)

    if model_cfg.get("freeze_vision_encoder", False):
        for parameter in model.encoder.parameters():
            parameter.requires_grad = False

    return model

def count_parameters(model: torch.nn.Module) -> dict[str, int]:
    total = sum(parameter.numel() for parameter in model.parameters())
    trainable = sum(parameter.numel() for parameter in model.parameters() if parameter.requires_grad)
    return {"total": total, "trainable": trainable}

def empty_vision_encoder_decoder_config() -> VisionEncoderDecoderConfig:
    return VisionEncoderDecoderConfig()

```

src/railcap/predict.py

```

from __future__ import annotations

import argparse
import json
from pathlib import Path
from typing import Any

import torch
from torch.utils.data import DataLoader
from tqdm import tqdm
from transformers import AutoImageProcessor, AutoTokenizer, VisionEncoderDecoderModel

from railcap.config import load_config
from railcap.data.dataset import RailwayCaptionDataset
from railcap.train import collate_batch
from railcap.utils.runtime import ensure_dir, pick_device

```

```

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Generate railway image captions.")
    parser.add_argument("--config", default="configs/default.yaml", type=Path)
    parser.add_argument("--checkpoint", type=Path, help="Model checkpoint directory.")
    parser.add_argument("--manifest", type=Path, help="Override manifest path.")
    parser.add_argument("--output", type=Path, help="Output JSONL path.")
    return parser.parse_args()

def _generation_kwargs(config: dict[str, Any]) -> dict[str, Any]:
    return dict(config.get("model", {}).get("generation", {}))

def main() -> None:
    args = parse_args()
    cfg = load_config(args.config)
    raw = cfg.raw

    checkpoint = args.checkpoint or (cfg.output_dir / "final")
    if not checkpoint.exists():
        raise FileNotFoundError(f"checkpoint not found: {checkpoint}")

    manifest_path = args.manifest or Path(raw["data"]["test_manifest"])
    output_path = args.output or (cfg.output_dir / "predictions.jsonl")
    ensure_dir(output_path.parent)

    image_processor = AutoImageProcessor.from_pretrained(checkpoint)
    tokenizer = AutoTokenizer.from_pretrained(checkpoint)
    model = VisionEncoderDecoderModel.from_pretrained(checkpoint)
    device = pick_device()
    model.to(device)
    model.eval()

    dataset = RailwayCaptionDataset(
        manifest_path,
        image_root=raw["data"].get("image_root", "."),
        image_processor=image_processor,
        tokenizer=None,
    )
    loader = DataLoader(
        dataset,
        batch_size=int(raw.get("eval", {}).get("batch_size", 8)),
        shuffle=False,
        num_workers=int(raw["data"].get("num_workers", 4)),
        collate_fn=collate_batch,
    )

    generation_kwargs = _generation_kwargs(raw)
    with output_path.open("w", encoding="utf-8") as f, torch.no_grad():
        for batch in tqdm(loader, desc="predict"):
            pixel_values = batch["pixel_values"].to(device)
            generated_ids = model.generate(pixel_values=pixel_values, **generation_kwargs)
            captions = tokenizer.batch_decode(generated_ids, skip_special_tokens=True)
            for sample, prediction in zip(batch["samples"], captions, strict=True):
                record = {
                    "image": sample.image,
                    "prediction": prediction.strip(),
                    "reference": sample.caption,
                    "equipment": sample.equipment,
                    "faults": sample.faults,
                    "risks": sample.risks,
                }
                f.write(json.dumps(record, ensure_ascii=False) + "\n")

    print(f"wrote predictions to {output_path}")

if __name__ == "__main__":
    main()

```

src/railcap/train.py

```
from __future__ import annotations

import argparse
from pathlib import Path
from typing import Any

import torch
from torch.utils.data import DataLoader
from tqdm import tqdm

from railcap.config import load_config
from railcap.data.dataset import RailwayCaptionDataset
from railcap.models.modeling import build_model, build_processors, count_parameters
from railcap.utils.runtime import ensure_dir, pick_device, set_seed


def collate_batch(batch: list[dict[str, Any]]) -> dict[str, Any]:
    output: dict[str, Any] = {
        "samples": [item["sample"] for item in batch],
        "captions": [item["caption"] for item in batch],
    }
    tensor_keys = ["pixel_values", "labels", "decoder_attention_mask"]
    for key in tensor_keys:
        if key in batch[0]:
            output[key] = torch.stack([item[key] for item in batch])
    return output


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Train railway image captioning model.")
    parser.add_argument("--config", default="configs/default.yaml", type=Path)
    return parser.parse_args()


def main() -> None:
    args = parse_args()
    cfg = load_config(args.config)
    raw = cfg.raw
    set_seed(int(raw.get("seed", 42)))
    output_dir = ensure_dir(cfg.output_dir)

    image_processor, tokenizer = build_processors(raw)
    model = build_model(raw, tokenizer)
    device = pick_device()
    model.to(device)

    params = count_parameters(model)
    print(f"parameters total={params['total']:,} trainable={params['trainable']:,}")

    data_cfg = raw["data"]
    train_dataset = RailwayCaptionDataset(
        data_cfg["train_manifest"],
        image_root=data_cfg.get("image_root", "."),
        image_processor=image_processor,
        tokenizer=tokenizer,
        max_caption_length=int(data_cfg.get("max_caption_length", 96)),
    )
    train_loader = DataLoader(
        train_dataset,
        batch_size=int(raw["train"].get("batch_size", 8)),
        shuffle=True,
        num_workers=int(data_cfg.get("num_workers", 4)),
        collate_fn=collate_batch,
    )

    optimizer = torch.optim.AdamW(
        (parameter for parameter in model.parameters() if parameter.requires_grad),
        lr=float(raw["train"].get("learning_rate", 5e-5)),
    )
```

```

        weight_decay=float(raw["train"].get("weight_decay", 0.01)),
    )

    use_amp = bool(raw["train"].get("mixed_precision", True)) and device.type == "cuda"
    scaler = torch.cuda.amp.GradScaler(enabled=use_amp)
    grad_accum_steps = int(raw["train"].get("grad_accum_steps", 1))
    log_every = int(raw["train"].get("log_every", 20))

    model.train()
    global_step = 0
    for epoch in range(int(raw["train"].get("epochs", 3))):
        progress = tqdm(train_loader, desc=f"epoch {epoch + 1}")
        optimizer.zero_grad(set_to_none=True)
        for step, batch in enumerate(progress, start=1):
            pixel_values = batch["pixel_values"].to(device)
            labels = batch["labels"].to(device)

            with torch.cuda.amp.autocast(enabled=use_amp):
                outputs = model(pixel_values=pixel_values, labels=labels)
                loss = outputs.loss / grad_accum_steps

            scaler.scale(loss).backward()
            if step % grad_accum_steps == 0:
                scaler.step(optimizer)
                scaler.update()
                optimizer.zero_grad(set_to_none=True)
                global_step += 1

            if global_step and global_step % log_every == 0:
                progress.set_postfix(loss=f"{loss.item() * grad_accum_steps:.4f}")

        if raw["train"].get("save_every_epoch", True):
            checkpoint_dir = output_dir / f"checkpoint-epoch-{epoch + 1}"
            model.save_pretrained(checkpoint_dir)
            tokenizer.save_pretrained(checkpoint_dir)
            image_processor.save_pretrained(checkpoint_dir)

    final_dir = output_dir / "final"
    model.save_pretrained(final_dir)
    tokenizer.save_pretrained(final_dir)
    image_processor.save_pretrained(final_dir)
    print(f"saved model to {final_dir}")

if __name__ == "__main__":
    main()

```

src/railcap/utils/__init__.py

```

"""Runtime utilities."""

```

src/railcap/utils/runtime.py

```

from __future__ import annotations

import random
from pathlib import Path

import torch

def set_seed(seed: int) -> None:
    random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

def ensure_dir(path: str | Path) -> Path:

```

```

directory = Path(path)
directory.mkdir(parents=True, exist_ok=True)
return directory

def pick_device() -> torch.device:
    return torch.device("cuda" if torch.cuda.is_available() else "cpu")

def cuda_memory_mb() -> float:
    if not torch.cuda.is_available():
        return 0.0
    return torch.cuda.max_memory_allocated() / 1024 / 1024

```

pyproject.toml

```

[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "railcap"
version = "0.1.0"
description = "Knowledge-guided lightweight image captioning for railway inspection scenarios."
readme = "README.md"
requires-python = ">=3.10"
dependencies = [
    "pillow>=10.0.0",
    "pyyaml>=6.0.1",
    "torch>=2.1.0",
    "transformers>=4.40.0",
    "tqdm>=4.66.0",
]

[project.optional-dependencies]
dev = [
    "pytest>=8.0.0",
    "ruff>=0.4.0",
]

[project.scripts]
railcap-train = "railcap.train:main"
railcap-predict = "railcap.predict:main"
railcap-evaluate = "railcap.evaluate:main"
railcap-benchmark = "railcap.benchmark:main"
railcap-build-manifest = "railcap.data.build_manifest:main"

[tool.setuptools.packages.find]
where = ["src"]

[tool.ruff]
line-length = 100
target-version = "py310"

[tool.pytest.ini_options]
pythonpath = ["src"]
testpaths = ["tests"]

```

configs/default.yaml

```

seed: 42
output_dir: outputs/default

data:
    train_manifest: data/train.jsonl
    val_manifest: data/val.jsonl
    test_manifest: data/test.jsonl
    image_root: .

```

```

max_caption_length: 96
num_workers: 4

knowledge:
  ontology_path: data/ontology/railway_terms.yaml
  enforce_terms: true
  hallucination_check: true

model:
  # Good starting point for a lightweight baseline. Replace with local models if offline.
  vision_encoder: google/vit-base-patch16-224-in21k
  text_decoder: distilgpt2
  pretrained_vision_encoder_decoder: null
  freeze_vision_encoder: false
  use_lora: false
  generation:
    max_new_tokens: 80
    num_beams: 3
    no_repeat_ngram_size: 3

train:
  epochs: 3
  batch_size: 8
  learning_rate: 5.0e-5
  weight_decay: 0.01
  grad_accum_steps: 1
  mixed_precision: true
  log_every: 20
  save_every_epoch: true

distillation:
  enabled: false
  teacher_checkpoint: null
  alpha_ce: 0.7
  alpha_kl: 0.3
  temperature: 2.0

compression:
  quantize_dynamic: false
  export_onnx: false
  onnx_path: outputs/default/model.onnx

eval:
  batch_size: 8
  metrics:
    - equipment_recall
    - fault_f1
    - risk_accuracy
    - hallucination_rate

```

tests/test_domain_metrics.py

```

from railcap.data.schema import CaptionSample
from railcap.knowledge.ontology import RailwayOntology
from railcap.metrics.domain_metrics import compute_domain_metrics

def test_compute_domain_metrics():
    ontology = RailwayOntology(
        equipment=("钢轨", "扣件"),
        faults=("扣件松动",),
        risks=("设备失效风险",),
        aliases={},
    )
    samples = [
        CaptionSample(
            image="a.jpg",
            caption="",
            equipment=["钢轨", "扣件"],

```

```

        faults=["扣件松动"],
        risks=["设备失效风险"],
    )
]
metrics = compute_domain_metrics(samples, ["钢轨和扣件可见，存在扣件松动和设备失效风险"], ontology)

assert metrics.equipment_recall == 1.0
assert metrics.fault_f1 == 1.0
assert metrics.risk_accuracy == 1.0
assert metrics.hallucination_rate == 0.0

```

tests/test_ontology.py

```

from railcap.knowledge.ontology import RailwayOntology

def test_extract_terms_with_aliases(tmp_path):
    ontology_path = tmp_path / "terms.yaml"
    ontology_path.write_text(
        """
equipment:
- 钢轨
faults:
- 异物侵限
risks:
- 行车安全风险
aliases:
  钢轨:
  - 铁轨
  异物侵限:
  - 异物侵入限界
    """.strip(),
        encoding="utf-8",
    )
    ontology = RailwayOntology.from_yaml(ontology_path)

    assert ontology.extract_terms("铁轨旁存在异物侵入限界，可能造成行车安全风险") == {
        "钢轨",
        "异物侵限",
        "行车安全风险",
    }

```